Multiple Comparators for Custom Sorting

Name: U.JAYASUDHA

Roll No.: 192411228

Course code: CSA0926

Submission Info

Question: C02-Q36

GitHub : https://github.com/ummadisettijayasudha136-ship-it/CSA0926.git

Submission Date: 1/8/2026

Problem Understanding

What We Need to Build

- Create a **Student** class with name and marks attributes
- Implement **two Comparators**: one for alphabetical sorting by name, another for numerical sorting by marks
- Demonstrate sorting the **same list** using both Comparators to show different orderings

Approach & Design Notes

Approach	Design notes
Classes used	Student(POJO),comparator interface,collection.sort()
NameComparator	Implement comparator<student>,uses string.compareTo() for alphabetical order
MarkedComparator	Implements comparator<student>,uses Integer comparison for ascending marks
Alternative	Lambda expressions for inline,concise comparator definition without separate class

Source Code

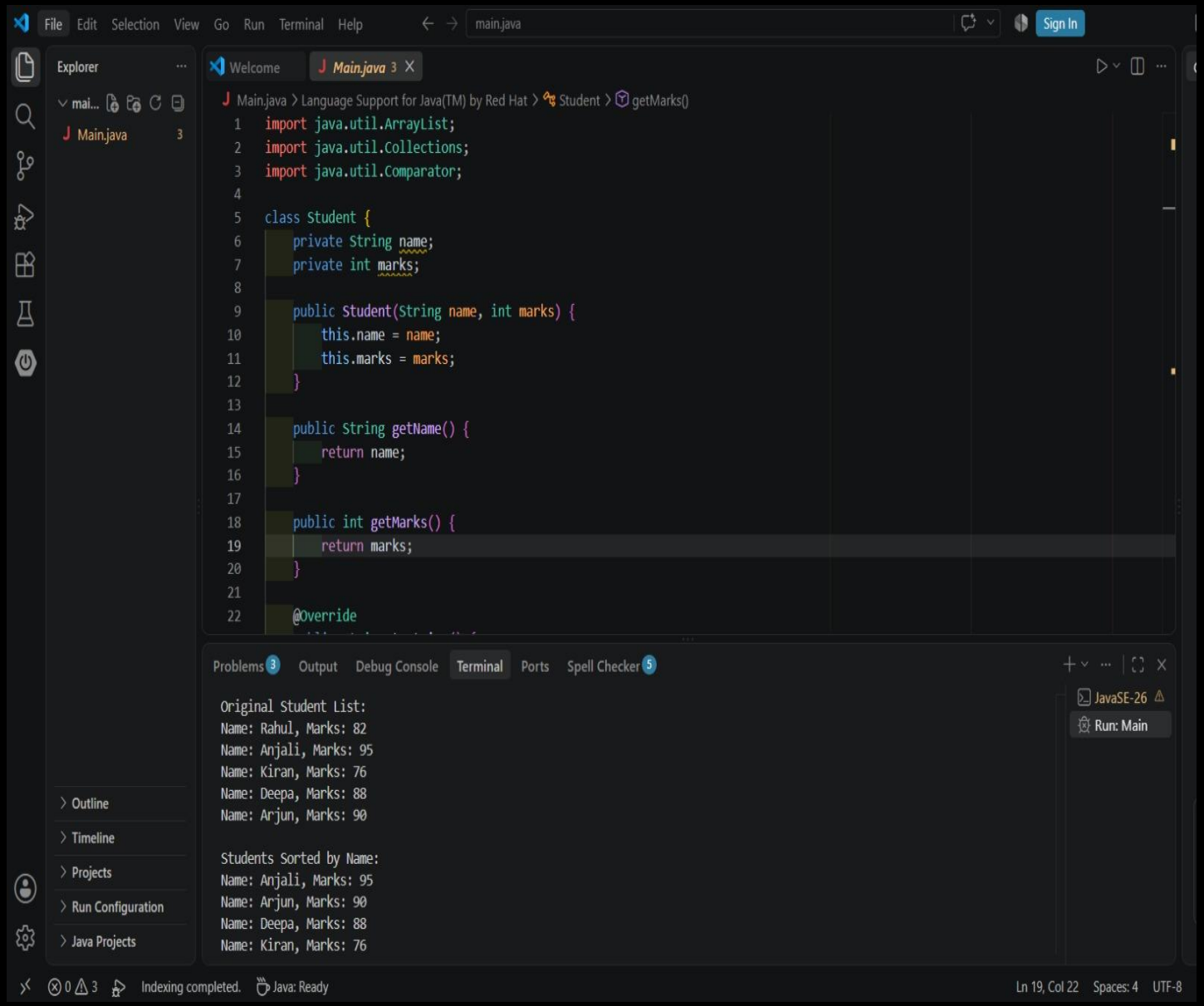
```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
class Student {
    private String name;
    private int marks;
    public Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
    public String getName() {
        return name;
    }
    public int getMarks() {
        return marks;
    }
    @Override
    public String toString() {
        return "Student{Name='" + name + "', Marks=" + marks + "}";
    }
}
class NameComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.getName().compareToIgnoreCase(s2.getName());
    }
}
class MarksComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return Integer.compare(s1.getMarks(), s2.getMarks());
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        ArrayList<Student> students = new ArrayList<>();
        students.add(new Student("Rahul", 82));
        students.add(new Student("Anjali", 95));
        students.add(new Student("Kiran", 76));
        students.add(new Student("Deepa", 88));
        students.add(new Student("Arjun", 90));

        // Original List
        System.out.println("Original List:");
        for (Student student : students) {
            System.out.println(student);
        }

        // Sort by Name
        Collections.sort(students, new NameComparator());

        System.out.println("\nStudents Sorted by Name:");
        for (Student student : students) {
            System.out.println(student);
        }

        // Sort by Marks
        Collections.sort(students, new MarksComparator());

        System.out.println("\nStudents Sorted by Marks:");
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Test Cases

1	2	3
Typical case	**Edge Case**	**Boundary Case**
Mixed student list sorted by name (A–Z) and marks (ascending) verifies both Comparators produce correct distinct orderings.	Students with the **same name** or **identical marks** verifies stability and tie-handling behavior of each Comparator.	**Single student** and **empty list** verifies no exceptions are thrown and output remains well-formed.

Screenshots of Execution



Reflection

Key Learnings

- Learned how **Comparator** enables flexible sorting without modifying the Student class — upholding the Open/Closed Principle
- Discovered that **lambda expressions** provide cleaner, more concise syntax than writing separate Comparator classes
- Understood the importance of **consistent comparison logic** for reliable, predictable sorting across all edge cases

Declaration

I declare that this assignment is my own work. I completed the program by understanding the given question and verified the output before submission. AI assistance was used only to understand the problem, improve the documentation, and organize the report. I confirm that the final submission is my original work.